

API

PSIU-v1 HTTP API

All endpoints are served by the Mongoose web server (Core/mongoose/) on port 80 (HTTP). Examples below use `psiu.local` (mDNS hostname `psiu`); substitute the device's IP if mDNS doesn't resolve on your network.

Authentication

Two accounts are hardcoded in `glue_authenticate` (`mongoose_glue.c`):

User Password Level

admin	admin	7
user	user	3

Send credentials with HTTP Basic Auth on every request — no separate login call is required for scripting:

```
curl -u admin:admin ...
```

Important quirk: every `/api/*` endpoint and `/websocket` requires *some* valid login, even ones whose `read_level/write_level` are 0. The dispatcher (`mongoose_impl.c`, `http_ev_handler`) unconditionally rejects requests with no authenticated session for anything matching `/api/#`, regardless of level — there's no "level 0 = public" bypass for that path. The only truly public endpoints are the hand-written custom handlers listed below (`/uid`, `/samplerate`, `/status`, `/wav`, `/audio.wav`, `/dtmf`), which carry an explicit bypass (`ch_public` in `http_ev_handler`).

A browser session can avoid resending the password on every request by calling `POST /api/login` once (with Basic Auth) to receive a `Set-Cookie: access_token=... session cookie (24 h)`. `POST /api/logout` clears it. Scripts can just keep using `-u` instead.

Public endpoints (no login required)

GET /uid

Device identity and network/loop profiling counters (e.g. for diagnosing throughput). Read-only.

```
curl http://psiu.local/uid
```

```
{"uid": "...", "loops": 123456, "ms": 98765, "poll_us": ..., "poll_max": ...,  
  "rd_us": ..., "rd_cnt": ..., "tcpip_us": ..., "evpoll_us": ..., "write_us": ...,  
  "drvtx_us": ..., "drvtx_cnt": ..., "iosend_us": ..., "evwrite_us": ...,  
  "txcopy_us": ..., "txcsum_us": ..., "nsent": ..., "nerr": ..., "ndrop": ...}
```

GET /samplerate, POST /samplerate

Read or switch the live audio sample rate (96 kHz / 192 kHz). Refuses while recording.

```
curl http://psiu.local/samplerate
```

```
curl -X POST http://psiu.local/samplerate -d '{"hz": 96000}'
```

```
{"ok": true, "rate": 96000, "recording": false, "supported": [96000, 192000]}
```

- 400 — unsupported hz value
- 409 — recording in progress, rate not changed

GET /status

Consolidated unit status: device UID, uptime, sample rate, recording state, input relay position, buffer wrap count, recorder/NAND health, recording counter.

```
curl http://psiu.local/status
```

```
{"uid": "...", "uptime_ms": ..., "sample_rate_hz": 192000, "recording": false,
  "xlr": true, "buffer_count": ..., "recorder_state": "idle",
  "pages_written": ..., "dropped_halves": ..., "bad_block_count": ...,
  "dma_errors": ..., "i2s_errors": ..., "recording_count": ...}
```

recorder_state is one of: idle, invalidate, pre_erase, wait_half, busy, erasing, finalize, full, error.

GET /wav

Metadata of the last completed NAND recording — the file /audio.wav would stream — without downloading it.

```
curl http://psiu.local/wav
```

```
{"available": true, "recording": false, "sample_rate_hz": 192000,
  "channels": 2, "bits": 32, "data_bytes": ..., "duration_ms": ...,
  "dropped_halves": ..., "prog_failures": ..., "recording_count": ...}
```

- 409 {"available": false, "recording": true} — a new recording is in progress (metadata still describes the *previous* file)
- 404 {"available": false, "recording": false} — no recording has ever completed (or the on-flash metadata layout predates a firmware update that bumped RECMETA_VERSION)

GET /audio.wav

Streams the last completed recording as a 44-byte-header PCM WAV file. Supports HTTP Range requests (resume-capable: curl -C -, wget -c, browsers).

```
curl -o psiu.wav http://psiu.local/audio.wav
curl -C - -o psiu.wav http://psiu.local/audio.wav # resume
```

- 409 recording in progress — same rule as /wav
- 404 no recording — nothing completed yet
- 416 — Range request past end of file

GET /dtmf

Live DTMF decoder state, for trigger calibration. Magnitudes reported as $10 \cdot \log_{10}(\text{mag}^2)$ integers (avoids float printf).

```
curl http://psiu.local/dtmf
```

```
{"frames": ..., "overruns": ..., "recording": 0, "symbol": ..., "digit": "-",
  "row": ..., "col": ..., "start_state": ..., "stop_state": ...,
  "starts": ..., "stops": ..., "mag_db": [..., ..., ..., ..., ..., ..., ..., ...]}
```

Authenticated endpoints (/api/*)

All require -u admin:admin or -u user:user (or a session cookie from /api/login) regardless of the level shown — see the auth quirk above. Reads are GET /api/<name>; writes are POST /api/<name> with a JSON body of the changed fields (unset fields keep their current value). The response body always echoes the full current state after the write.

Live, hardware-backed

inputsel — input relay (XLR/RCA)

read 0 / write 0

```
curl -u admin:admin http://psiu.local/api/inputsel
curl -u admin:admin -X POST http://psiu.local/api/inputsel -d '{"xlr":true}'
```

`{"xlr":true}` — relay energized = XLR selected; false = RCA.

buffer — audio buffer wrap counter

read 0 / write 0, read-only in practice

```
curl -u admin:admin http://psiu.local/api/buffer
```

`{"count":...}` — increments ~187/s at 192 kHz stereo.

sampling — recording on/off (toggle form)

read 0 / write 0

```
curl -u admin:admin -X POST http://psiu.local/api/sampling -d '{"running":true}'
curl -u admin:admin -X POST http://psiu.local/api/sampling -d '{"running":false}'
```

`{"running":true}` while a recording is in progress.

samplestart / samplestop — recording on/off (action form)

read 0 / write 0

```
curl -u admin:admin -X POST http://psiu.local/api/samplestart -d 'true'
curl -u admin:admin -X POST http://psiu.local/api/samplestop -d 'true'
```

Body just needs to be non-empty to trigger; content is ignored. Same underlying effect as the sampling toggle (Recorder_Start()/Recorder_Stop()).

firmware_update — OTA upload

read 3 / write 7 (admin only)

```
curl -u admin:admin -X POST http://psiu.local/api/firmware_update \
  --data-binary @PSIU-v1.bin
```

Streams the raw .bin as the POST body. The device validates and applies the image, then reboots into it.

Mock / wizard scaffolding — not wired to real hardware

These are the Mongoose Wizard's demo/template data blocks, left in place but never connected to actual sensors or device state. Reading or writing them only affects an in-RAM mock struct (mongoose_glue.c); they're listed here so they aren't mistaken for real telemetry.

Endpoint	Level (r/w)	Notes
debug	0/0	<code>{"debugstring":"..."}</code> — round-trips, no effect
signal	0/0	<code>{"R":0,"L":0}</code> — static, not live audio level metering
state	0/0, read- only	<code>{"speed","temperature","humidity","uptime","version","online","lights","level"}</code> — fixed demo values

Endpoint	Level (r/w)	Notes
leds	3/3	{"led1","led2","led3"} — unrelated to the physical LBLUE1/LBLUE2/LRED pins, which are driven directly by firmware logic (see CLAUDE.md / recorder & Ethernet-activity code), not this API
network_settings	3/7	{"ip_address","gw_address","netmask","dhcp"} — display only; the real network config comes from MX_ETH_Init/DHCP, not this struct
settings	3/7	generic demo fields (string_val, log_level, double_val, int_val, bool_val)
security	7/7	{"admin_password","user_password"} — does not change real login credentials ; those are hardcoded in glue_authenticate
reboot	3/7	action; fakes a 1 s “in progress” state, does not actually reset the MCU
reformat	3/7	action; fakes a 1 s “in progress” state, does not actually erase the NAND
save_event	3/7	action; fakes a 1 s “in progress” state, no real effect
loglevels	0/0, read-only	static list ["disabled","error","info","debug","verbose"]
events	0/0, read-only	static two-entry demo event log

Session / system

Endpoint	Method	Purpose
/api/login	GET/POST + Basic Auth	Exchange credentials for a Set-Cookie: access_token=... session (24 h)
/api/logout	GET/POST	Clear the session cookie
/api/ok	GET	Liveness check, returns true (still requires login, per the auth quirk)
/api/heartbeat	GET	{"version":N} — bumps whenever any writable API field changes, for UI change-polling
/websocket	GET (upgrade)	WebSocket upgrade for the web UI’s live updates; also requires login

Endpoint summary

Path	Auth	Live?
GET /uid	public	yes
GET/POST /samplerate	public	yes
GET /status	public	yes
GET /wav	public	yes
GET /audio.wav	public	yes
GET /dtmf	public	yes
GET/POST /api/inputsel	login	yes
GET/POST /api/buffer	login	yes (read)
GET/POST /api/sampling	login	yes
POST /api/samplestart	login	yes
POST /api/samplestop	login	yes
POST /api/firmware_update	login (admin)	yes

Path	Auth	Live?
GET/POST /api/debug	login	mock
GET/POST /api/signal	login	mock
GET /api/state	login	mock
GET/POST /api/leds	login (3)	mock
GET/POST /api/network_settings	login (3/7)	mock
GET/POST /api/settings	login (3/7)	mock
GET/POST /api/security	login (7)	mock
POST /api/reboot	login (3/7)	mock
POST /api/reformat	login (3/7)	mock
POST /api/save_event	login (3/7)	mock
GET /api/loglevels	login	mock
GET /api/events	login	mock
/api/login, /api/logout, /api/ok, /api/heartbeat, /websocket	login	system